
OPEN3D DESDE PYTHON:

INTRODUCCIÓN AL MODELADO 3D

INTRODUCCIÓN

En los proyectos de esta asignatura, se deberá modelar al menos un objeto virtual. Para simular un entorno virtual existen muchos simuladores que se podrían utilizar. Open3D es un entorno de simulación sencillo con funcionalidad para tratar los tipos de representación de datos 3D más utilizados.

Open3D es una biblioteca de código abierto para el procesamiento de datos en 3D. Haz clic [aquí](#) para abrir la documentación de Open3D. Esta biblioteca ofrece funcionalidades para:

- leer y escribir archivos en [formatos estándar](#) que contienen datos en 3D;
- preprocesar datos en 3D para recortar, reducir ruido, muestrear a menor resolución, etc.;
- procesar datos en 3D para registro, detección, segmentación, etc.;
- y [visualizar](#) datos en 3D.

OBJETIVOS

El objetivo principal de esta sesión de prácticas es aprender a modelar objetos virtuales simples en movimiento utilizando Open3D desde Python.

MATERIALES

Para desarrollar esta práctica, descargue los siguientes materiales a una carpeta de trabajo local (en adelante, “[working_folder_path](#)”) en el ordenador donde vaya a ejecutar el cuaderno (*.ipynb) o el fichero Python (*.py).

- PoliformaT -> Recursos/2-Software-&-cuadernos/40-Jupyter lab Tutorial – Geometry
- PoliformaT -> Recursos/2-Software-&-cuadernos/41-Jupyter lab Tutorial - Data structures
- PoliformaT -> Recursos/2-Software-&-cuadernos/42-PyCharm demo - Dinamic scene

Importante >> No olvide hacer una copia de seguridad del código implementado (por ejemplo, en su carpeta W:) al finalizar la práctica

1.- ARRANQUE CON Jupyter lab

Si nunca has utilizado PyCharm, es mejor empezar con Jupyter lab. Ejecuta el programa Jupyter lab desde un terminal “Anaconda Prompt (miniconda3)”, tras activar el “environment” adecuado.

```
(base) C:\windows\system32> conda activate open3d_env
(open3d_env) C:\Users\username> jupyter lab --notebook-dir “working_folder_path”
```

Después de arrancar Jupyter lab, escribe en una célula de código los comandos:

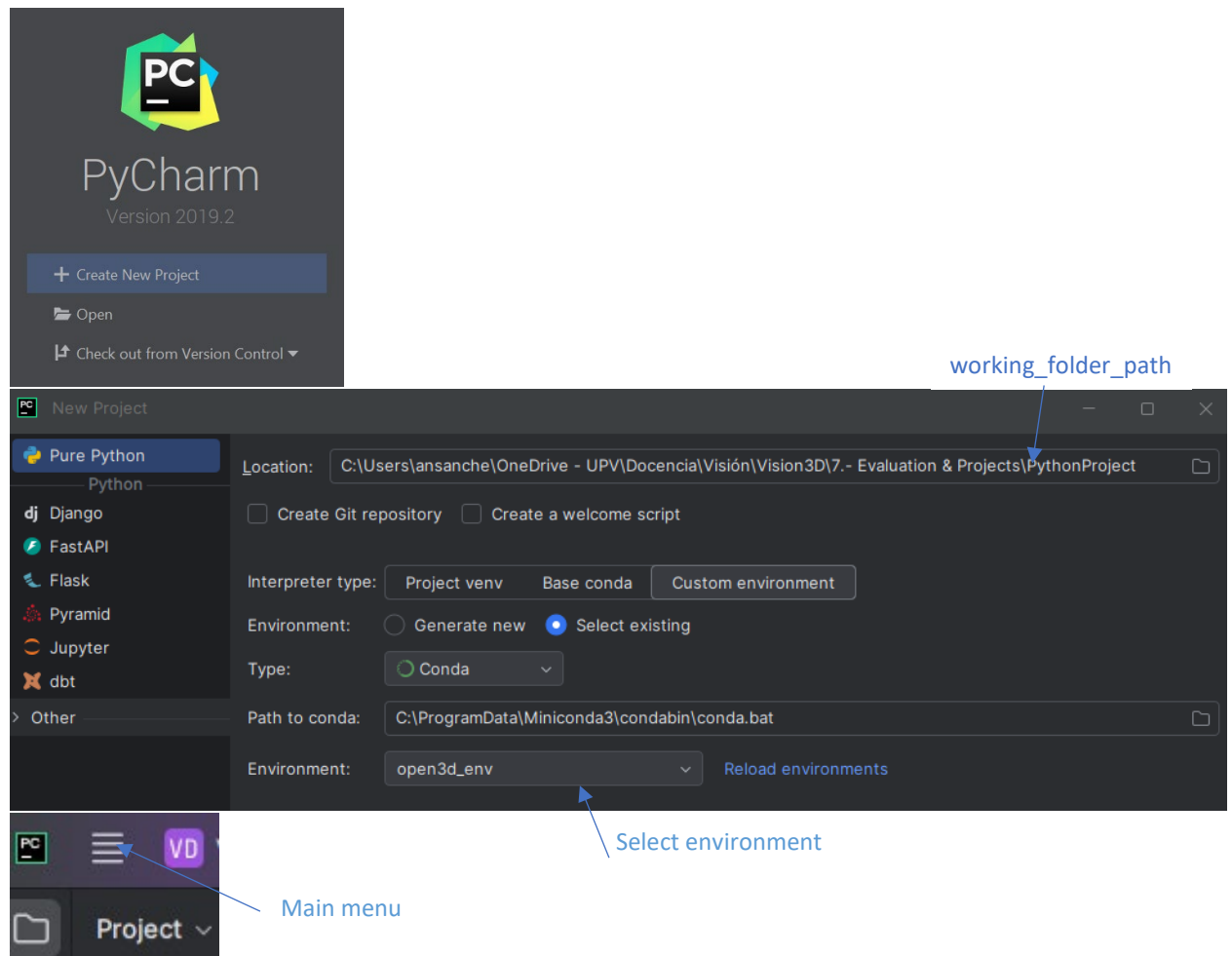
```
import open3d as o3d
import utilities as ut
from utilities import *
```

Mediante módulo open3d, Python encontrará todos los métodos de Python que sirven de interfaz entre la librería Open3D de C++ y Python. Además, la librería utilities contiene una serie de funcionalidades auxiliares.

Utilidad >> Para habilitar el autocompletado de código en JupyterLab, presione la tecla Tab mientras escribe el código.

1.2.- ARRANQUE CON PyCharm

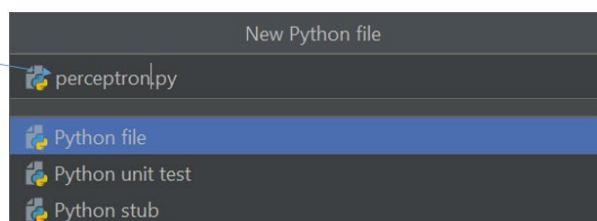
Si prefieres comenzar a trabajar con PyCharm, deberás crear un proyecto y seleccionar el “environment” adecuado desde el desplegable “Environment”.



Para crear un nuevo fichero Python (*.py) simplemente selecciona la siguiente opción del “Menu principal”:

- Main menú -> File -> New -> Python File

New file name



2.- MODELADO DE OBJETOS

En el cuaderno “40_Geometry.ipynb” muestra ejemplos de creación de modelos simples utilizando la funcionalidad de open3D. Por ejemplo:

- la creación de una rejilla del suelo (en el plano $z=0$), (analizar la implementación en la función `XY_grid` de la librería `utilities.py`):

```
ground_grid = ut.XY_grid((-10,-9), (10,9), (1,1))
```

- el sistema de coordenadas del mundo denominado `world_frame`:

```
# Creating world Cartesian coordinate frame.
world_frame = o3d.geometry.TriangleMesh.create_coordinate_frame(
    size=1,
    origin=[0, 0, 0])
world_frame.compute_vertex_normals()
```

Para los objetos tipo malla será necesario calcular las normales para que posteriormente se puedan visualizar estos objetos.

- una esfera denominada `p`:

```
p = o3d.geometry.TriangleMesh.create_sphere(radius=0.06)
p.translate([2, 3, 1])
p.paint_uniform_color([0.5, 0.5, 0.5])
p.compute_vertex_normals()
```

- un cubo en modelado alámbrico, denominado `box`:

```
# Creating a wire box and the point p.
box = ut.create_wire_cube(width=2, height=3, depth=1)
box.paint_uniform_color([0.7, 0., 0.])
```

En este cuaderno también se muestran ejemplos de visualización de una escena (definida como una lista de objetos) en una ventana:

```
# Define the scene as a list of objects.
# In this case the contains the world_frame and the ground_grid.
scene_list = [ground_grid, world_frame]

# Visualizing the scene in a window. Press ESC to close the window.
o3d.visualization.draw_geometries(scene_list, left=0, top=30)
```

En el cuaderno “41_Data_Structures.ipynb” muestra ejemplos de cómo leer objetos desde ficheros con formato `*.obj`, cómo añadir una textura a un cubo, cómo leer nubes de puntos de ficheros con formato `*.ply` o cómo convertir un objeto modelado con mallas triangulares a una nube de puntos.

- Leer un cubo desde fichero del tipo `.obj` (analizar la función `read_obj` en la librería de `utilities.py`):

```
## Reading the 3D model file as a 3D mesh using open3d.
path_obj = ".\\data\\Cube.obj"
mesh = ut.read_obj(path_obj)
```

Finalmente, el cuaderno “41_Create_Cube.ipynb” describe como escribir un fichero `*.obj` que contiene un cubo con textura desde cero, partiendo de la definición de sus vértices, caras y textura.

3.- SIMULACIÓN DINÁMICA

El script de Python “DinamicVirtualScene.py” crea una escena 3D interactiva que muestra:

- Un sistema de coordenadas (como referencia).
- Una esfera que se mueve continuamente en pequeñas cantidades a lo largo de la diagonal (ejes x, y, z).

La cámara está posicionada utilizando parámetros almacenados en un archivo de calibración, lo que da una vista realista y controlada de la escena.

Este script realiza una simulación del movimiento de un objeto virtual simple, como es una esfera, que se mueve a una velocidad constante predefinida. En cada iteración de la simulación, se actualiza la posición del objeto y se visualiza en una ventana gráfica que actúa como visor del objeto en movimiento, permitiendo así visualizar su trayectoria.

4.-TAREA

Deberéis crear y mostrar una escena virtual simple que sea un gemelo digital de la escena de la aplicación a desarrollar en vuestro proyecto. Se deberá desarrollar un visor que muestre la escena virtual utilizando la funcionalidad de Open3D (o alternatively utilizar un simulador de robótica interactivo como por ejemplo RoboDK). Esta escena virtual deberá incluir al menos un objeto real y uno virtual.